03. functions

```
Data Sciences Institute
Python
```

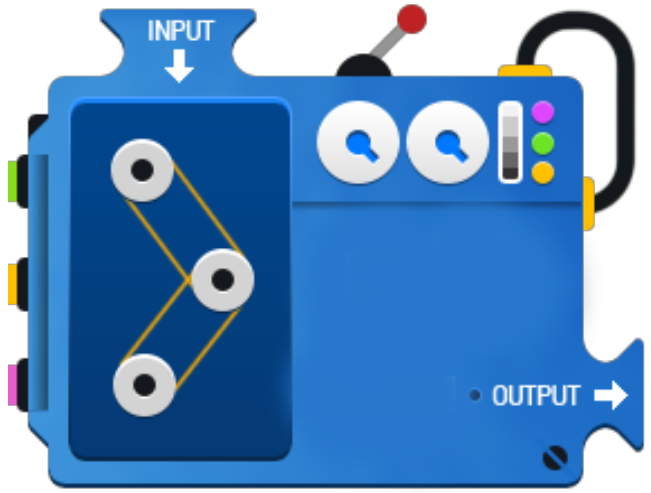
Learning objectives

- Understand what functions are and why they are useful
- Use common built-in functions
- Nest and combine functions
- Write your own functions
- Document functions using docstrings
- Understand function structure and variable scope
- Apply a basic function design recipe

What is a function?

- A **function** is a block of code that performs a task.
  - It can **take inputs** and **return outputs**.
  - Functions help make code **organized** and **reusable**.

- Functions can have zero or more inputs.
  - Inputs are called **arguments**.
- Functions usually **return a value after processing**



How to call a function

- To use a function, we **call** it.
- Syntax: `function_name()`
- Python comes with several functions already **built-in**.
- To display information, for example, we can call the `` `print()` `` function.

```
print(42)        # Output: 42
```

Here, print is the function being *called*

```
print(10 ** 2) # Output: 100
```

Note, we can also pass an expression as an argument!

More built-in functions

- Python has many other **built-in functions** you can use right away.
- These functions make common tasks easier.
  - Use `abs()` to get the absolute value of a number.

```
abs(-42)                # Output: 42
```

  - Use `round()` to round a number to the nearest integer (or specified decimal places).

```
round(3.7)              # Output: 4
round(3.14159, 2)       # Output: 3.14
```

  - Convert values using `int()` (integer) or `float()` (decimal)

```
int(3.7)                # Output: 3
float(5)                # Output: 5.0
```

Nesting functions

- You can **use functions inside other functions—**this is called **nesting**.
- Nested functions are **evaluated from the inside out**.

```
print(abs(round(-2 * 1.75)))
```

- This allows us to perform **multiple operations** in a single line making code **compact and efficient!**

Evaluation order:

1. `-2 * 1.75` → multiplication happens first
   - Result: -3.5
2. `round()` → rounds the result
   - Result: -4
3. `abs()` → converts to absolute value
   - Result: 4
4. `print()` → displays the final output
   - Output: 4

How to Learn About a Function

- Sometimes we need to know:
  - What a function **does**
  - What **arguments** it can take

- Two main ways to find out:
  1. **Online documentation**
     https://docs.python.org/3/library/functions.html
  2. **Built-in help function**
     `help()` shows information about a function directly in Python.

```
help(function_name)
```

Example of online documentation

round(*number, ndigits=None*)

Return *number* rounded to *ndigits* precision after the decimal point. If *ndigits* is omitted or is `None`, it returns the nearest integer to its input.

For the built-in types supporting `round()`, values are rounded to the closest multiple of 10 to the power minus *ndigits*; if two multiples are equally close, rounding is done toward the even choice (so, for example, both `round(0.5)` and `round(-0.5)` are `0`, and `round(1.5)` is `2`). Any integer value is valid for *ndigits* (positive, zero, or negative). The return value is an integer if *ndigits* is omitted or `None`. Otherwise, the return value has the same type as *number*.

For a general Python object `number`, `round` delegates to `number.__round__`.

Note: The behavior of `round()` for floats can be surprising: for example, `round(2.675, 2)` gives `2.67` instead of the expected `2.68`. This is not a bug: it's a result of the fact that most decimal fractions can't be represented exactly as a float. See Floating-Point Arithmetic: Issues and Limitations for more information.

This tells us what the function does and what arguments it takes!

Example of help function

```
help(round)
```

```
Help on built-in function round in module builtins:

round(number, ndigits=None)
    Round a number to a given precision in decimal
    digits.
    The return value is an integer if ndigits is
    omitted or None. Otherwise, the return value has
    the same type as the number. ndigits may be
    negative.
```

This also tells us what the function does and what arguments it takes, without having to leave Python!

Writing your own function

- We can also create our own functions, which allows us to reuse code without copying and pasting!

1. In Python, we define a function using the `def` keyword. This tells Python that we are creating a new function.
2. Function name comes right after def.
   - Function names:
     - Can include letters, numbers, and underscores
     - Cannot start with a number
     - Are case sensitive
     - Usually start with a lowercase letter

```
def function_name
```

Writing your own function

3. After the function name, we list any **parameters inside parentheses**.
    - **A parameter is a variable that exists only inside the function**. When we call a function and pass in values (called *arguments*), those values are assigned to the parameters.

```
def function_name(parameter = argument)
```

4. The line defining the function ends with a colon (:).

- The **function body is the indented block of code** that follows. This code:
    - Runs only when the function is called, not when it is defined
    - Can include comments and multiple statements

```
def function_name(parameter = argument):
    code to execute when function is called
```

Writing your own function

5. The function usually ends with a return statement.

- **A return statement specifies what value the function will send back** when it's called
  - *Note*. If you don't write a return statement, Python automatically returns `None` (a special value that represents "no data")

```
def function_name(parameter = argument):
    code to execute when function is called
    return
```

Example

- Let's create a function that converts degrees from Celsius to Fahrenheit

  The formula is (**°C** × 9/5) + 32 = **°F**

```
def c_to_f(degrees_c):

    #convert degrees from Celsius to Fahrenheit
     return (degrees_c * (9/5)) + 32
```

- defines the function name with 1 parameter, `degrees_c`
- `degrees_c` only exists inside the function
- converts the Celsius value from `degrees_c` to farenheit using the standard formula and returns the result!

Example

- Once we've defined our variable, we can use it!

```
c_to_f(0) # returns 32.0
c_to_f(100) # returns 212.0
```

The function does nothing until it is *called*. When called, it takes the input value, performs the calculation, and returns the converted temperature.

Example

- We can save the output of a function by assigning it to a variable

```
freezing_f = c_to_f(0)
freezing_f # output 32.0
```

This allows us to store the returned value (32.0) in the variable freezing_f

Exercise # 1

- Try this on your own:
    - Write a function that converts Fahrenheit to Celsius

      The formula is (**°F** – 32) × 5/9 = **°C**

Multiple parameters

- So far, the functions we've seen have had a single parameter. However, functions can have multiple parameters.
- When a function has more than one parameter, the order of the arguments matters.
  - The first argument is assigned to the first parameter
  - The second argument is assigned to the second parameter
  - And so, on
- Because Python matches arguments to parameters **by position**, passing values in the wrong order can lead to incorrect results.

Example

- Let's create a function that has two parameters: a divisor and a dividend.

```
def divide(dividend, divisor):
    return dividend / divisor
```

This function:

- Takes two parameters
- Divides the **dividend** by the **divisor**
- Returns the result

```
divide(0,2)# output 0
divide(2,0)# output error division by zero
```

Optional parameters and default arguments

- Functions can have **default values** for their parameters.
  - This is useful when a value is commonly used, because it allows us to **omit that argument** when the default works for our needs.
- If we want to use a different value, we can **override the default** by explicitly passing in an argument.

Example

- Let's create a function that calculates the sale tax on a price.

```
def calc_sales_tax(price, tax_rate=0.13):
     return price * tax_rate
```

`price` is a required parameter → you must always provide it.
`tax_rate` is an optional parameter with a default value of 0.13 (Ontario's sales tax rate).
If no value is given for `tax_rate`, Python automatically uses 0.13.

```
calc_sales_tax(100)# default rate 13%, returns 13.0
calc_sales_tax(100, .05)# rate of 5%, returns 5.0
```

Keyword arguments

- When a function has multiple optional parameters, we can use **keyword arguments** to specify exactly which parameter a value should be applied to.
  - Using a keyword argument looks like assigning a value to a parameter name
  - This makes the code easier to read and avoids mistakes from incorrect argument order

Example

- Let's create a function that calculates the total bill cost with price, tax rate and tip rate.

```
def calc_total_bill(price, tax_rate=0.13, tip_rate=0.2):
    tax = price * tax_rate
    tip = price * tip_rate
    return price + tax + tip
```

`price` is a required parameter → you must always provide it.
`tax_rate` is an optional parameter with a default value of 0.13
`tip_rate` is an optional parameter with a default value of 0.20
If no value is given for `tax_rate` and `tip_rate`, Python automatically uses 0.13 and 0.20, respectively.

Example

default tax and tip rate

```
calc_total_bill(100)# returns 133.0
```

default tax but specify tip rate

```
calc_total_bill(100, tip_rate = 0.22)# returns 135.0
```

- Here, we use the `tip_rate` keyword argument to specify that the second argument is not tax rate, but tip rate!
- If we didn't, it would assume we were setting the `tax_rate` to 0.22

```
calc_total_bill(100, 0.22)# returns 142.0
```

Documenting functions

- We can use the built-in `help()` function to see information about **both built-in and user-defined functions**.
- **Docstrings**
  - To get useful information for user-defined functions, we add a docstring.
  - A docstring is **a special string that describes what a function does**.
  - It is placed immediately after the function definition line.
  - Docstrings are typically written with triple quotes: ''' or """.
  - When we call **`help()` on a function, Python displays its docstring**.

Example

```
def c_to_f(degrees_c):
    """
    Convert a temperature from Celsius to Fahrenheit.

    Parameters:
    degrees_c (float): Temperature in degrees Celsius.

    Returns:
    float: Temperature in degrees Fahrenheit.
    """
    return (degrees_c * (9/5)) + 32
```

Example

- Now when we call the help function…

```
help(c_to_f)

Help on function c_to_f in module __main__:
c_to_f(degrees_c)
    Convert a temperature from Celsius to Fahrenheit.
    Parameters:
    degrees_c (float): Temperature in degrees Celsius.
    Returns:
    float: Temperature in degrees Fahrenheit.
```

Review of function structure

```
def function_name(parameters):

    """
    docstring
    """



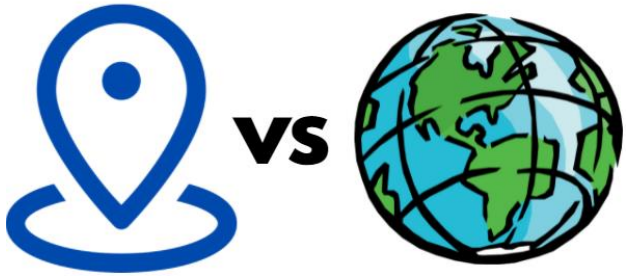
    function body
    return
```

function header → tells us the function name and parameters

docstring→ describes what the function does, accepted arguments, what gets returned etc.

function body → code that runs when function is called, ends with a return statement specifying the output

Variable scope



- We mentioned earlier that **parameters are variables used only inside functions**. This means:
  - If you try to access a parameter like `degrees_c` **outside** the function `c_to_f()`, Python will raise a `NameError`.

Local scope	Global scope
Parameters are **locally scoped**, meaning they exist **only inside the function**.	Variables defined **outside of any function** have **global scope**.
Scope refers to the **context in which a variable can be accessed**.	They can be accessed from **anywhere in the program**, including inside functions (though modifying them inside a function requires extra care).
Local means the variable is accessible **only within the function** where it is defined.	

Local variable example

- This is a variable we defined inside a function

```
def example():
    local_var = 10
    return local_var
```

- If we tried to call this variable out of the function, we'd get a NameError!

```
local_var
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[1], line 1
----> 1 local_var

NameError: name 'local_var' is not defined
```

Global variable example

- This is a variable we defined outside of a function

```
global_var = 5
```

- We can access this variable inside and outside of a function

```
def example():
    return global_var
```

```
example()# returns 5
```

```
global_var # output: 5
```

Exercise # 2

- Create a function on your own:
    - Think about what your function does. Come up with a few examples of inputs and outputs.
    - Think of a meaningful function name.
    - Write thorough function documentation.
    - Code the function body.
    - Test the function against the examples from step 1.